Laboratorio 1.2

Probabilidades, Bayes, Embeddings y Stacking

Dr. Juan Bekios Calfa

Deep Learning

Universidad Católica del Norte

Recordando el laboratorio anterior

- Se trabajó con modelos de clasificación supervisada como regresión logística, árboles de decisión, SVM y KNN.
- Se discutieron métricas de evaluación tales como accuracy, precision, recall y F1-score.
- Se observó que un solo modelo no siempre captura toda la estructura del problema.
- También se introdujo la idea de pipeline y la necesidad de evaluar correctamente con validación.

Transición al laboratorio 1.2

Ahora el foco está en comprender por qué las **probabilidades** son útiles, cómo usar **Bayes** en este problema, qué papel cumplen los **embeddings** y cómo combinar modelos mediante **stacking**.

Propósito del laboratorio 1.2

Objetivo general

Construir una comprensión conceptual y práctica de los métodos de ensamble tipo stacking aplicados a la predicción del deterioro cognitivo, integrando fundamentos probabilísticos, uso de Bayes, representación mediante embeddings y una implementación reproducible en Visual Studio Code.

Objetivos específicos

- Reforzar el contexto del problema de deterioro cognitivo.
- Introducir probabilidad y Bayes para clasificación.
- Explicar embeddings como nueva representación de información.
- Implementar stacking paso a paso.
- Trabajar con Conda, VS Code y GitHub de forma organizada.

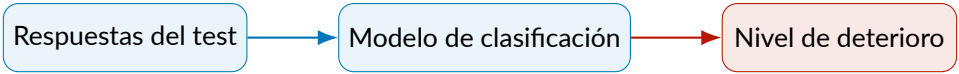
Problema de aplicación: deterioro cognitivo

- Se desea predecir el nivel de deterioro cognitivo a partir de variables obtenidas desde pruebas neuropsicológicas.
- Los atributos pueden incluir memoria, orientación, lenguaje, atención y otras dimensiones del desempeño cognitivo.
- La salida es una **clase** o categoría de deterioro, por ejemplo usando una codificación de la escala GDS.
- Desde el punto de vista computacional, esto se modela como un problema de **clasificación supervisada multiclase**.

Dificultades metodológicas

Pocas muestras, clases desbalanceadas y posibilidad de sobreajuste. Por eso no basta con entrenar un modelo: es necesario diseñar bien el experimento.

Estructura conceptual del problema



- Las respuestas del paciente se transforman en variables numéricas o categóricas.
- El modelo aprende relaciones entre dichas variables y la clase objetivo.
- En este laboratorio, además de predecir, interesa comprender **cómo** el modelo decide.

¿Por qué introducir probabilidades?

- Una predicción de clase por sí sola puede ocultar incertidumbre.
- Dos pacientes pueden ser clasificados en la misma clase, pero con niveles de confianza muy distintos.
- Las probabilidades permiten interpretar mejor la decisión del modelo.
- También son muy útiles como entrada para un segundo modelo en stacking.

Ejemplo intuitivo

No es lo mismo predecir clase 1 con probabilidad 0.51 que con probabilidad 0.95. En ambos casos la clase es la misma, pero la seguridad del modelo cambia mucho.

Concepto básico de probabilidad

La probabilidad cuantifica la incertidumbre de un evento:

$$P(A) = \frac{\text{casos favorables}}{\text{casos posibles}}$$

La probabilidad cuantifica la incertidumbre de un evento:

$$P(A) = \frac{\text{casos favorables}}{\text{casos posibles}}$$

En Machine Learning, una idea típica es estimar:

$$P(y = k \mid x)$$

- x representa los atributos del paciente.
- $y = k$ representa la clase k de deterioro cognitivo.

Interpretación

El modelo estima qué tan probable es que una observación pertenezca a cada clase posible.

Clasificación probabilística en la práctica

Clase	Probabilidad
Sin deterioro	0.10
Deterioro leve	0.20
Deterioro moderado	0.55
Deterioro severo	0.15

El modelo elegiría **deterioro moderado**, pero además informa el grado de incertidumbre relativo con respecto a las otras clases.

Idea clave

Cuando el problema es sensible, como en un contexto clínico o cognitivo, las probabilidades suelen ser más informativas que una etiqueta sola.

Teorema de Bayes

$$P(C \mid X) = \frac{P(X \mid C)P(C)}{P(X)}$$

- C: clase objetivo, por ejemplo un nivel de deterioro.
- X: conjunto de características observadas.
- $P(C \mid X)$: probabilidad posterior de la clase dada la evidencia.
- $P(X \mid C)$: probabilidad de observar esas características si la clase es C.
- $P(C)$: probabilidad previa de la clase.
- $P(X)$: probabilidad total de observar la evidencia.

Bayes en el problema de deterioro cognitivo

Queremos estimar:

$$P(\text{GDS} = k \mid x_1, x_2, \ldots, x_n)$$

- $x_1, x_2, \ldots, x_n$ pueden representar respuestas o puntajes del test cognitivo.
- La clase k representa una categoría de deterioro cognitivo.
- Se comparan las probabilidades posteriores para todas las clases posibles.
- La clase con mayor probabilidad es la predicción final.

Lectura intuitiva

El modelo pregunta: *dadas estas respuestas, ¿qué tan probable es cada nivel de deterioro?*

¿Por qué se llama Naive Bayes?

Se llama **naive** porque asume independencia condicional entre variables dadas la clase:

$$P(X \mid C) = \prod_{i=1}^{n} P(x_i \mid C)$$

- Esta suposición simplifica mucho el cálculo.
- En la práctica, muchas variables cognitivas sí están correlacionadas.
- Aun así, el modelo suele funcionar razonablemente bien y es una buena línea base.
- En stacking puede aportar una perspectiva distinta a otros clasificadores.

Ejemplo conceptual de Bayes

- Suponga dos atributos simples: Memoria = baja y Orientación = baja.
- Considere dos clases: Clase 0 = sin deterioro y Clase 1 = con deterioro.
- Si ambas evidencias son más frecuentes en la clase 1, la probabilidad posterior favorecerá esa clase.
- Bayes combina la evidencia parcial de varias variables para decidir qué clase es más plausible.

Conclusión

Bayes combina evidencia parcial de varias variables para decidir qué clase es más plausible.

Ventajas de Bayes en este laboratorio

- Es simple de entender e implementar.
- Entrena rápido incluso con pocos datos.
- Puede entregar probabilidades por clase.
- Es útil como baseline para comparar contra modelos más complejos.
- Sus probabilidades de salida pueden utilizarse como entrada de un meta-modelo en stacking.

Importante

No se usa porque sea siempre el mejor modelo, sino porque ayuda a introducir una forma probabilística de pensar el problema.

Ejemplo sencillo con GaussianNB

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB

# Ejemplo simple de datos
# Cada fila representa un paciente
# label = clase objetivo

df = pd.DataFrame({
    'memoria': [8, 7, 3, 2, 9, 1, 6, 2],
    'orientacion': [9, 8, 4, 3, 9, 2, 7, 1],
    'lenguaje': [8, 7, 5, 4, 9, 3, 6, 2],
    'atencion': [7, 8, 4, 3, 8, 2, 6, 1],
    'label': [0, 0, 1, 1, 0, 1, 0, 1]
})

X = df.drop(columns='label')
y = df['label']
```

Interpretación del ejemplo anterior

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42
)

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_prob = model.predict_proba(X_test)

print("Predicciones:", y_pred)
print("Probabilidades:")
print(y_prob)
```

Si la salida incluye algo como:

$$[0{,}85, 0{,}15] \qquad [0{,}20, 0{,}80]$$

- Paciente 1: 85 % para clase 0 y 15 % para clase 1.

Interpretación del ejemplo anterior

Si la salida incluye algo como:

$$[0{,}85, 0{,}15] \qquad [0{,}20, 0{,}80]$$

- Paciente 1: 85 % para clase 0 y 15 % para clase 1.
- Paciente 2: 20 % para clase 0 y 80 % para clase 1.

Aprendizaje esperado

Se debe distinguir entre **clase predicha** y **probabilidades asociadas a cada clase**.

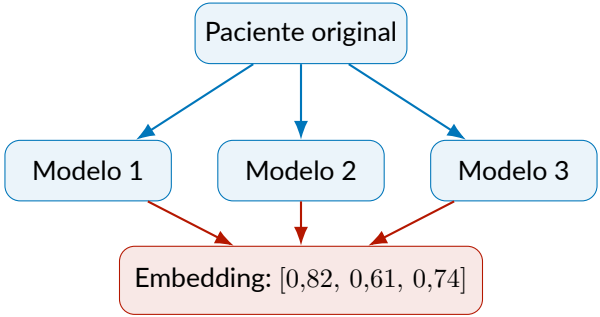
Embeddings: idea general

- Un **embedding** es una representación vectorial de información en un espacio numérico.
- En texto, una palabra puede representarse por un vector.
- En imágenes, una red neuronal puede producir un vector de características.
- En stacking, las salidas de varios modelos pueden formar un nuevo vector para representar cada muestra.

Interpretación en este laboratorio

Aquí usaremos la palabra embedding en un sentido práctico: una nueva representación construida a partir de predicciones o probabilidades de los modelos base.

Embeddings a partir de modelos



Cada modelo aporta una salida. Juntas forman una representación nueva que luego puede usar un meta-modelo.

¿Qué es stacking?

- Stacking es un método de ensamble en dos niveles.
- **Nivel 0:** modelos base que aprenden desde los atributos originales.
- **Nivel 1:** meta-modelo que aprende a combinar las salidas de los modelos base.

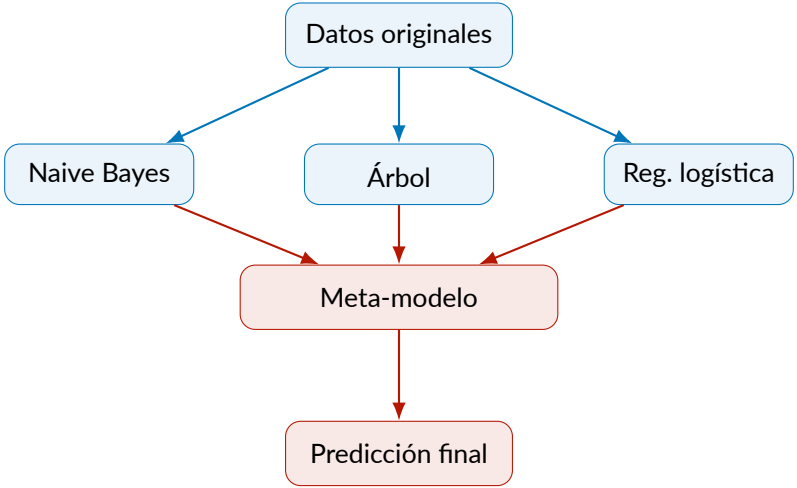
Idea intuitiva

En lugar de elegir un único modelo, se deja que varios modelos opinen y se entrena otro modelo para decidir cómo combinar esas opiniones.

Ventaja

Puede capturar fortalezas complementarias entre modelos lineales, no lineales y probabilísticos.

Arquitectura general de stacking



¿Por qué stacking puede ayudar?

- Los modelos no se equivocan siempre en las mismas muestras.
- Algunos capturan relaciones lineales; otros, patrones no lineales.
- Un clasificador probabilístico como Naive Bayes puede complementar a un árbol o una regresión logística.
- El meta-modelo aprende cuándo confiar más en cada modelo base.

Lectura pedagógica

Stacking no garantiza automáticamente mejores resultados, pero ofrece una forma flexible de combinar distintas perspectivas sobre el mismo problema.

Diseño correcto de stacking

- Las predicciones usadas para entrenar el meta-modelo no deben provenir del mismo ajuste directo sobre los mismos datos sin control.
- Idealmente, las salidas del nivel 0 deben obtenerse mediante validación interna u out-of-fold predictions.
- Esto reduce fuga de información y evita optimismo artificial.
- En un laboratorio introductorio se puede mostrar primero una versión simple y luego discutir la versión metodológicamente correcta.

Advertencia

Un stacking mal implementado puede parecer muy bueno en entrenamiento y decepcionar en evaluación real.

Implementación simple de modelos base

```
from sklearn.naive_bayes import GaussianNB
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression

nb = GaussianNB()
tree = DecisionTreeClassifier(random_state=42)
lr = LogisticRegression(max_iter=1000)

nb.fit(X_train, y_train)
tree.fit(X_train, y_train)
lr.fit(X_train, y_train)
```

- Se usan tres clasificadores con comportamientos distintos.
- La variedad entre modelos base es importante para que el stacking tenga sentido.
- Si todos los modelos son demasiado similares, el beneficio del ensamble puede ser menor.

Construcción del embedding para el meta-modelo

```
import numpy as np

p1 = nb.predict_proba(X_train)[:, 1]
p2 = tree.predict_proba(X_train)[:, 1]
p3 = lr.predict_proba(X_train)[:, 1]

meta_X = np.column_stack((p1, p2, p3))
```

- Cada columna representa la salida probabilística de un modelo base.
- La nueva matriz `meta_X` es el embedding utilizado por el meta-modelo.
- En un problema multiclase, puede utilizarse una probabilidad por clase y por modelo, o estrategias equivalentes.

Entrenamiento del meta-modelo

```
meta_model = LogisticRegression(max_iter=1000)
meta_model.fit(meta_X, y_train)
```

- El meta-modelo aprende cómo combinar las probabilidades anteriores.
- Puede usar reglas simples o relaciones más complejas, según el algoritmo escogido.
- En esta versión se usa regresión logística por ser interpretable y fácil de explicar.

Idea clave

Los modelos base ya no trabajan sobre los atributos originales en esta etapa; el meta-modelo trabaja sobre un nuevo espacio de representación.

Predicción final con stacking

```
p1_test = nb.predict_proba(X_test)[:, 1]
p2_test = tree.predict_proba(X_test)[:, 1]
p3_test = lr.predict_proba(X_test)[:, 1]

meta_X_test = np.column_stack((p1_test, p2_test, p3_test))
y_pred = meta_model.predict(meta_X_test)
```

- Primero se generan las salidas de los modelos base en el conjunto de prueba.
- Después se construye el embedding del test.
- Finalmente, el meta-modelo entrega la predicción final.

Interpretación conceptual del stacking

- Si Naive Bayes acierta mejor en ciertas clases, el meta-modelo puede aprender a darle más peso allí.
- Si el árbol responde mejor a patrones no lineales, su probabilidad también queda representada.
- El meta-modelo no reemplaza a los modelos base: aprende a coordinarlos.

Mensaje para la clase

Stacking puede verse como una conversación entre varios modelos, donde un modelo final escucha a todos y aprende a sintetizar la mejor respuesta.

Preprocesamiento y fuga de información

- Toda transformación dependiente de los datos debe ajustarse solo con datos de entrenamiento.
- Ejemplos: escalamiento, imputación, selección de variables, balanceo.
- Si estas transformaciones se aplican antes de separar correctamente entrenamiento y prueba, aparece **fuga de información**.
- La fuga produce resultados demasiado optimistas y comparaciones poco confiables.

Regla metodológica

Lo correcto es encapsular las transformaciones dentro de un pipeline o dentro de la lógica de validación.

Validación y comparación justa

- Si el conjunto de datos es pequeño, no conviene evaluar con una única partición fija.
- Se recomienda validación cruzada estratificada para preservar mejor las proporciones de clase.
- Todos los modelos deben evaluarse con el mismo protocolo.
- Solo así la comparación entre modelos base y stacking será justa.

Consecuencia práctica

La validez de la comparación depende tanto del diseño experimental como del algoritmo utilizado.

Métricas recomendadas

- Accuracy
- Precision macro
- Recall macro
- F1-score macro
- Matriz de confusión

Cuidado

En problemas desbalanceados, accuracy puede esconder mal desempeño en clases minoritarias.

Recomendación

No quedarse con una sola métrica. La interpretación debe considerar rendimiento global y comportamiento por clase.

Ejemplo de interpretación de matriz de confusión

- La diagonal principal representa aciertos.
- Los valores fuera de la diagonal muestran confusiones entre clases.
- En un problema clínico, puede ser más relevante evitar ciertos errores que otros.
- Por eso la matriz de confusión complementa las métricas globales.

Clase real \ Clase predicha				
	12	2	1	0
	1	9	2	0
	0	2	8	1
	0	1	2	6

Implementación en Visual Studio Code

- Visual Studio Code permite trabajar con proyectos, terminal integrada, control de versiones y extensiones de Python.
- Es una buena opción para que el laboratorio tenga estructura de proyecto y no solo scripts aislados.
- Se recomienda instalar las extensiones de Python y Jupyter si se trabajará con notebooks.

Meta formativa

No solo se busca que el código funcione, sino que esté organizado de manera mantenible y reproducible.

Creación del entorno virtual con Conda

```
conda create -n lab_stacking python=3.10
conda activate lab_stacking
pip install numpy pandas scikit-learn matplotlib jupyter
```

- El entorno virtual aísla dependencias del proyecto.
- Esto evita conflictos con otras librerías instaladas en el sistema.
- También facilita la reproducibilidad del laboratorio en distintos equipos.

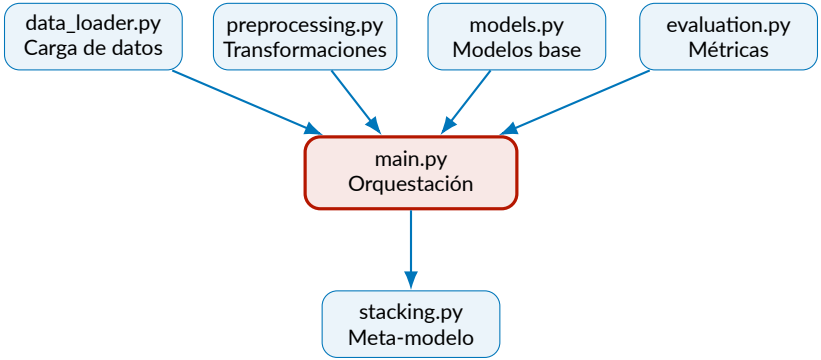
Selección del intérprete en VS Code

1. Abrir la carpeta del proyecto.
2. Presionar `Ctrl + Shift + P`.
3. Buscar `Python: Select Interpreter`.
4. Elegir el intérprete asociado al entorno `lab_stacking`.

Resultado esperado

A partir de ese momento, la terminal integrada y la ejecución de scripts usarán el entorno correcto.

Estructura modular recomendada del proyecto



Separar responsabilidades hace más claro el experimento y facilita modificar partes del pipeline sin desordenar todo el proyecto.

Jerarquía sugerida de directorios

Estructura

```
laboratorio_1_2/
|-- data/
  |-- raw/
  |-- processed/
  `-- outputs/
|-- notebooks/
  `-- exploracion.ipynb
|-- src/
  |-- main.py
  |-- data_loader.py
  |-- preprocessing.py
  |-- models.py
  |-- stacking.py
  `-- evaluation.py
|-- environment.yml
|-- README.md
`-- .gitignore
```

Lectura

- `data/`: datos y salidas.
- `notebooks/`: exploración rápida.
- `src/`: implementación principal.
- `README.md`: instrucciones del proyecto.

Inicialización del repositorio Git

```
git init
git add .
git commit -m "Inicializacion del laboratorio 1.2"
```

- Git permite registrar cambios a lo largo del desarrollo.
- Es recomendable hacer commits pequeños y descriptivos.
- Así se puede volver a versiones anteriores y entender la evolución del proyecto.

Conexión del proyecto con GitHub

```
git remote add origin https://github.com/usuario/repositorio.git
git branch -M main
git push -u origin main
```

- GitHub facilita respaldo, colaboración y seguimiento del trabajo.
- También es útil para compartir el laboratorio con ayudantes o compañeros de equipo.
- En un curso, tener el repositorio ordenado ayuda a transparentar el proceso de desarrollo.

Buenas prácticas para el desarrollo del laboratorio

- Separar datos, código y resultados.
- Documentar decisiones importantes en el README o en el informe.
- Mantener nombres claros para archivos, variables y funciones.
- Evitar repetir código cuando una función puede encapsular una tarea.
- Registrar dependencias y versiones de librerías.

Mensaje pedagógico

Una buena solución no solo da resultados; también puede ser leída, entendida, reproducida y mantenida por otros.

Secuencia metodológica sugerida

1. Comprender el problema y el dataset.
2. Revisar distribución de clases.
3. Diseñar división o validación del experimento.
4. Entrenar un modelo Bayes simple para entender probabilidades.
5. Construir modelos base diversos.
6. Generar embeddings desde sus salidas.
7. Entrenar el meta-modelo de stacking.
8. Comparar con modelos individuales.
9. Interpretar resultados y errores.
10. Documentar el flujo completo.

Actividades concretas del laboratorio

- Explicar con sus palabras el problema de deterioro cognitivo como clasificación supervisada.
- Implementar un ejemplo base con Naive Bayes.
- Mostrar probabilidades de salida e interpretarlas.
- Construir un ejemplo simple de stacking con al menos tres modelos base.
- Comparar un modelo individual contra el stacking.
- Organizar la solución en una estructura modular y subirla a GitHub.

Resultados esperados del aprendizaje

Al finalizar el laboratorio, el estudiante debiera ser capaz de:

- Explicar qué significa una probabilidad de clasificación.
- Describir cómo Bayes se aplica a un problema de deterioro cognitivo.
- Entender un embedding como una nueva representación útil para combinar modelos.
- Implementar un stacking básico en Python.
- Configurar un entorno reproducible en VS Code con Conda y GitHub.

Errores comunes que se deben evitar

- Interpretar la clase predicha como si fuera certeza absoluta.
- Usar probabilidades del entrenamiento de forma ingenua para el meta-modelo sin discutir fuga de información.
- Comparar modelos con protocolos distintos.
- Escalar o transformar datos fuera del pipeline de evaluación.
- Entregar un proyecto sin estructura o sin documentación mínima.

Idea fuerza

Los errores metodológicos pueden arruinar una buena implementación algorítmica.

Entregables sugeridos

- Código fuente modular del laboratorio.
- Repositorio GitHub con estructura ordenada.
- Un breve informe o notebook explicativo.
- Resultados de evaluación y comparación entre modelos.
- Reflexión final sobre ventajas y limitaciones del stacking en este problema.

Criterios de evaluación propuestos

Criterio	%
Comprensión del problema y fundamentos probabilísticos	20
Uso de Bayes e interpretación de probabilidades	15
Implementación del stacking y claridad conceptual	25
Organización modular del código y reproducibilidad	20
Evaluación, comparación e interpretación de resultados	20

Observación

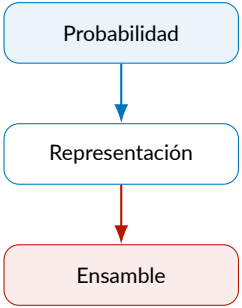
La parte conceptual y la parte práctica tienen el mismo peso formativo: no basta con que el código funcione si no se comprende qué está haciendo.

Bibliografía para profundizar

- Géron, A. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. Capítulos sobre clasificación y ensambles.
- Bishop, C. *Pattern Recognition and Machine Learning*. Capítulos sobre modelos probabilísticos.
- Hastie, Tibshirani y Friedman. *The Elements of Statistical Learning*. Secciones sobre ensemble learning.
- Documentación oficial de `scikit-learn` para `GaussianNB`, `StackingClassifier` y métricas de clasificación.

Cierre del laboratorio 1.2

- La probabilidad permite pasar de una clasificación rígida a una decisión con incertidumbre.
- Bayes ayuda a introducir una mirada probabilística simple y útil.
- Los embeddings permiten representar información de otra manera.
- El stacking combina modelos y puede aprovechar sus fortalezas complementarias.
- Una buena metodología experimental es tan importante como el modelo mismo.



¿Preguntas?